

Theories of Straight beams: Part 1

Braxton Lopez

16 February 2026

1 Vocabulary

Shape Functions Shape Functions are mathematical functions used to describe the geometry of an aerodynamic body. They are used to describe complex geometric features using few variables, as opposed to having to create thousands of mesh points. Typically, a solver will use discretization points (or nodes) to approximate a simulation, generally assuming the shape of the body to be linear and be represented by Eq. 1. As you create more nodes, your answer will become increasingly accurate. These linearized shape functions will generally follow the partition of unity property and the Kronecker delta property. For more information on these properties, see footnote 1¹.

$$N_i = a_i + b_i x \quad (1)$$

It is possible to make it converge faster and that is by increasing the polynomial order of the shape functions. Similar to the linear shape functions, quadratic shape functions can be introduced. The approximate solution is then not piecewise linear as we have seen so far, but piecewise quadratic².

$$N_i = a_i + b_i x + c_i x^2 \quad (2)$$

Shape functions can also be used to generally represent different 3D geometries for elements of a body, such as triangles, quadrilaterals, hexahedrals, tetrahedrals, etc.

Linearization Linearization is the approximation of an equation that defines any given system as a linear equation instead of its original more complicated, higher order equation. Instead of:

$$\dot{x} = f(x(t), u(t)) \quad (3)$$

you can express it using:

$$\dot{x} = Ax(t) + Bu(t) \quad (4)$$

In order to do this, you must replace a curved slope with a linear slope by finding the tangent line to the curve at a given point, and replacing the curve with the tangent line, then repeating this as many times as desired. This allows you to approximate the shape of the curve using only discretized linear portions. More precisely, find the first order approximation of the nonlinear equation around the given points, and use those discretized sections to make a linear equation.

Virtual Work The internal work integral:

$$\delta W_{int} = \int_{\Omega} \sigma : \delta \epsilon d\Omega \quad (5)$$

¹<https://teachbooks.tudelft.nl/computational-modelling/introduction/shape.html>

²<https://teachbooks.tudelft.nl/computational-modelling/introduction/shape.html>

represents the work that the internal stresses would exert to resist deformation if the structure were experiencing a virtual strain field (it was experiencing plastic/elastic deformation).

The two external work integrals:

$$\delta W_{ext} = \int_{\Omega} \delta u * d\Omega + \int_{\Gamma_N} \delta u * t d\Gamma \quad (6)$$

represent the work that would be done by virtual external sources. The first one represents the work done by virtual external force fields (like gravity or distributed loads throughout the beam), and the second one represents the work that would be done by virtual surface forces (such as applied loads, pressures, shear tractions, etc). Note that these forces are not actually applied to a physical beam, and that they are purely hypothetical descriptions of how it *should* behave under the forces/strains. If a beam is in static equilibrium, the external and internal works should balance:

$$\delta W_{ext} = \delta W_{int} \quad (7)$$

If unbalanced, the beam would accelerate in one direction (if $\delta W_{ext} > \delta W_{int}$, the structure would accelerate in a direction dictated by the external forces, and if $\delta W_{ext} < \delta W_{int}$, the beam begin returning to its original (undeformed) position.

Newton-Raphson Method The Newton-Raphson Method is an algorithm for finding roots of nonlinear functions. It uses tangent lines in order to rapidly locate the root and uses the equation:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (8)$$

You can find the root by determining where $x_{n+1} - x_n$ becomes so small that more iterations won't meaningfully change the answer. This method assumes a good initial guess and will fail if the initial guess is too far from the root. It can also fail if the derivative $f'(x)$ is either zero or extremely small, as this can trick a computational system into thinking it has found the root. This method does not provide extreme accuracy, and increasing the number of iterations increases the accuracy of the root. FEM uses this to update displacements.

2 Theory

i. Strong vs Weak Form Equations The strong form of a problem is a set of partial differential equations (or a single PDE) which describe the physics of the problem. Solving the strong form of the problem leads to the exact solution of the continuous system (note that it has to be continuous). However, for many engineering problems, it is not possible to obtain the exact solution, and numerical methods need to be employed to find an approximate solution³, which is known as the weak form. The weak form

³<https://teachbooks.tudelft.nl/computational-modelling/introduction/strong.html>, section 2.1

The strong Form of the 1D Poisson Equation is:

$$-\frac{\partial}{\partial x}(v(x)\frac{\partial u(x)}{\partial x}) = f(x) \quad (9)$$

where $v(x)$ is a material property, $u(x)$ the solution and $f(x)$ a source, all of which depend on the coordinate x . It can also be rewritten as:

$$-\frac{\partial}{\partial x}\Lambda + f = 0 \quad (10)$$

where Λ is a flux term, of which the interpretation depends on the problem at hand. In the case of heat conductivity, Λ is the heat flux, while in Darcy's law, Λ is the flow rate, and in extension of a rod, Λ is the stress. The typical constitutive relationship between the flux Λ and the unknown solution u is given by⁴:

$$\Lambda = -v\frac{\partial u}{\partial x} \quad (11)$$

In general, the weak form of the problem is derived by multiplying the strong form by a test function, integrating over the domain, and then applying integration by parts on the terms containing derivatives. In this manner, the order of derivatives appearing in the equations is reduced and the resulting form is symmetric (in general), facilitating a more efficient numerical solution⁵.

For example, when doing this to the strong form equation:

$$-EA\frac{\partial^2 u}{\partial x^2} = f, \quad (12)$$

the first step is pre-multiplication of left hand side and right hand side with a test function $w(x)$ and integrating over the domain. Assuming constant EA , we get:

$$-\int_0^L wEA\frac{\partial^2 u}{\partial x^2}dx = \int_0^L wfdx \quad (13)$$

where the equality has to hold for all possible test functions w . This equation reduces to:

$$\int_0^L \frac{\partial w}{\partial x}EA\frac{\partial u}{\partial x} = \int_0^L wfdx + w(L)F, \quad (14)$$

For a more complete derivation, see footnote 6⁶. In essence, the strong form equations are the exact solution and the weak form equations provide an approximation.

⁴<https://teachbooks.tudelft.nl/computational-modelling/introduction/strong.html>, section 2.1

⁵<https://teachbooks.tudelft.nl/computational-modelling/introduction/strong.html>, section 2.2

⁶<https://teachbooks.tudelft.nl/computational-modelling/introduction/weak.html>, section 2.2

ii. FEA Representations vs. Continuous Representations Shape Functions (FEA) are mathematical functions used to describe the geometry of an aerodynamic body. They are used to describe complex geometric features using few variables, as opposed to having to create thousands of mesh points. Typically, a solver will use discretization points (or nodes) to approximate a simulation, generally assuming the shape of the body to be linear and be represented by Eq. 1. As you create more nodes, your answer will become increasingly accurate. These linearized shape functions will generally follow the partition of unity property and the Kronecker delta property. For more information on these properties, see footnote 1.

A continuous representation defines every point along the domain and does *not* use linearization to approximate the general shape. A continuous representation uses partial order differentials in order to get an exact answer, while FEA approximation uses weak form equations to approximate the answer). This makes a continuous representation more accurate; however, its generally more computationally costly than a FEA representation. FEA aims to reduce the cost of these analysis's and allows for easier handling of complicated geometry.

iii. Matrix Formulation for Two-Node Beam Elements Each element within a Finite Element Method will contain geometric information, specifically the displacement u and the rotation θ . In 3D, it will contain 6 degrees of freedom: displacements u , v , and w , and rotations θ_x , θ_y , θ_z , in 2D it will only contain 4, and in 1D it will contain two degrees of freedom (u, θ_x). The purpose of these elements is to allow for bodies with more complex geometric features to be analyzed. Due to FEM's inability to use a continuous representation, you must approximate segments of the beam (see ii. of this document for more information) to get an approximate geometry. Each element will have its own information (as listed previously), and the beam's information is depicted by the sum of all the elements' information.

The matrix formulation for a linear two-node beam element is given by:

$$\{F\} = [K_e]\{d\} \quad (15)$$

where F is the force matrix, K_e is the stiffness matrix for the element, and d is the matrix containing the degrees of freedom (with one column total and one row for each degree of freedom. For a 2D beam in bending, it would follow the form:

$$\{d\} = \begin{bmatrix} u_1 \\ \theta_1 \\ u_2 \\ \theta_2 \end{bmatrix}$$

since a 2D beam only has 4 degrees of freedom. This means that Eq. 15 would look something like:

$$\begin{bmatrix} f_1 \\ q\Delta x \\ q\Delta x \\ \frac{1}{2}q\Delta x + F \end{bmatrix} = \frac{EA}{\Delta x} \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \begin{bmatrix} u_1 \\ \theta_1 \\ u_2 \\ \theta_2 \end{bmatrix}$$

For information on how to solve this, see this footnote⁷. This same matrix formulation holds true for a 3D beam element.

iv. Shape Functions (FEA) and Their Requirements Shape Functions are mathematical functions used to describe the geometry of an aerodynamic body. They are used to describe complex geometric features using few variables, as opposed to having to create thousands of mesh points. Typically, a solver will use discretization points (or nodes) to approximate a simulation, generally assuming the shape of the body to be linear and be represented by Eq. 1. As you create more nodes, your answer will become increasingly accurate. These linearized shape functions generally must follow the partition of unity property and the Kronecker delta property. For more information on these properties, see footnote 1.

It is possible to make it converge faster, and that is by increasing the polynomial order of the shape functions. Similar to the linear shape functions, quadratic shape functions can be introduced. The approximate solution is then not piecewise linear as we have seen so far, but piecewise quadratic⁸.

$$N_i = a_i + b_i x + c_i x^2 \quad (16)$$

Also note that these shape functions *must* be continuous over the domain.

v. Linear vs Nonlinear Effect Modeling In *iii. Matrix Formulation for Two-Node Beam Elements*, we discussed how matrix formulation can be used to model linear effects, specifically the relationship $\{F\} = [K_e]\{d\}$. This function is based on the assumption that the element has two nodes and all properties in the space between those two nodes follow a linear relationship. When we have a nonlinear effect (such as the stiffness increasing as deformation is increased), this relationship no longer holds true. Because these properties change, the equilibrium equations also change as the structure deforms, therefore you aren't solving the same structure you began with. The relationship between force and displacement is no longer proportional.

Its necessary to have this relationship relating forces and displacements to solve a model because the forces applied directly relate to how much a beam displaces/how much stress it undergoes. For nonlinear effects, A FEM will use the Newton-Raphson method to solve for

⁷<https://teachbooks.tudelft.nl/computational-modelling/introduction/matrix.html>, section 2.4 "Finite Element Implementation"

⁸<https://teachbooks.tudelft.nl/computational-modelling/introduction/shape.html>

the displacements, and that's why it requires the tangent stiffness (which is the derivative relationship between forces and displacements), represented by:

$$\frac{\partial f_{int}}{\partial u} \quad (17)$$

For more information on this method, see *Newton-Raphson Method* in Part 1 of this document. This allows you to find the displacements in a situation in which it is impossible to find the exact solutions every time. For a linear system, the solver can simply solve the relationship $\{F\} = [K_e]\{d\}$ since the stiffness of the beam doesn't vary. For nonlinear effects; however, the components of the stiffness matrix are variable at every point, so you require something more to solve it.

vi. Virtual Work and its Relations The internal work integral:

$$\delta W_{int} = \int_{\Omega} \sigma : \delta \epsilon d\Omega \quad (18)$$

represents the work that the internal stresses would exert to resist deformation *if* the structure were experiencing a virtual strain field (it was experiencing plastic/elastic deformation).

The two external work integrals:

$$\delta W_{ext} = \int_{\Omega} \delta u * d\Omega + \int_{\Gamma_N} \delta u * t d\Gamma \quad (19)$$

represent the work that would be done by virtual external sources. The first one represents the work done by virtual external force fields (like gravity or distributed loads throughout the beam), and the second one represents the work that would be done by virtual surface forces (such as applied loads, pressures, shear tractions, etc). Note that these forces are not actually applied to a physical beam, and that they are purely hypothetical descriptions of how it *should* behave under the forces/strains. If a beam is in static equilibrium, the external and internal works should balance:

$$\delta W_{ext} = \delta W_{int} \quad (20)$$

If unbalanced, the beam would accelerate in one direction (if $\delta W_{ext} > \delta W_{int}$, the structure would accelerate in a direction dictated by the external forces, and if $\delta W_{ext} < \delta W_{int}$, the beam begin returning to its original (undeformed) position.

vii. Derivation of $\frac{\partial f_{int}}{\partial u}$ The virtual work equation for the weak form is represented by the equation:

$$- \int_{\Omega} \nabla^s \delta u : \sigma d\Omega + \int_{\Omega} \delta u * b d\Omega + \int_{\Gamma_N} \delta u * t d\Gamma = 0 \quad (21)$$

By recognizing that the gradient of a virtual displacement can be seen as a virtual strain (i.e. $\nabla^s \delta u = \delta \epsilon$), we arrive at:

$$-\int_{\Omega} \delta \epsilon : \sigma d\Omega + \int_{\Omega} \delta u * b d\Omega + \int_{\Gamma_N} \delta u * t d\Gamma = 0 \quad (22)$$

where the first integral represents the internal virtual work, and the last two integrals represent the external virtual work. As discussed above, this shows the balance between internal work and external work. To reach the discrete form, you can use the relationship $\delta u = N\delta a$ and $\delta \epsilon = B\delta a$. Use

$$(AB)^T = B^T A^T \quad (23)$$

to remove δa from the equation. Doing this and rearranging the equation leaves us with:

$$\delta a^T \int_{\Omega} B^T \sigma d\Omega = \delta a^T \left(\int_{\Omega} N^T b d\Omega + \int_{\Gamma_N} N^T t d\Gamma \right) \quad (24)$$

where the integral on the left hand side of the equation is equivalent to f_{int} (not including the δa^T , and everything in the parenthesis represent f_{ext} . We know this because we can see the terms above being multiplied by the displacements, therefore the integrals must represent the forces. If you eliminate δa , it simplifies to:

$$f_{int}(a) = f_{ext} \quad (25)$$

where a is synonymous with the displacement u . The steps on how to derive this were found in this footnote⁹. Under linear assumptions, the general expression simplifies to:

$$\{F\} = [K_e]\{d\} \quad (26)$$

For nonlinear assumptions, this relationship doesn't hold true and you are required to find the tangent stiffness matrix. For information on why and how it affects how you solve the system, see *v. Linear vs Nonlinear Effect Modeling*, as well as how the tangent stiffness matrix simplifies for linear problems.

viii. Using Iterative Methods To Solve For Displacements There are two ways to solve for nodal displacements, one for nonlinear problems and one for linear problems. The following is for *linear* problems:

Each element within a Finite Element Method will contain geometric information, specifically the displacement u and the rotation θ . In 3D, it will contain 6 degrees of freedom: displacements u , v , and w , and rotations θ_x , θ_y , θ_z , in 2D it will only contain 4, and in 1D it will contain two degrees of freedom (u , θ_x). The purpose of these elements is to allow for bodies with more complex geometric features to be analyzed. Due to FEM's inability to use a continuous representation, you must approximate segments of the beam (see ii. of this document for more information) to get an approximate geometry. Each

⁹https://teachbooks.tudelft.nl/computational-modelling/solid_nonlinear/linearization.html, Section 5.2

element will have its own information (as listed previously), and the beam's information is depicted by the sum of all the elements' information.

The matrix formulation for a linear two-node beam element is given by:

$$\{F\} = [K_e]\{d\} \quad (27)$$

where F is the force matrix, K_e is the stiffness matrix for the element, and d is the matrix containing the degrees of freedom (with one column total and one row for each degree of freedom. For a 2D beam in bending, it would follow the form:

$$\{d\} = \begin{bmatrix} u_1 \\ \theta_1 \\ u_2 \\ \theta_2 \end{bmatrix}$$

since a 2D beam only has 4 degrees of freedom. This means that Eq. 15 would look something like:

$$\begin{bmatrix} f_1 \\ q\Delta x \\ q\Delta x \\ \frac{1}{2}q\Delta x + F \end{bmatrix} = \frac{EA}{\Delta x} \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \begin{bmatrix} u_1 \\ \theta_1 \\ u_2 \\ \theta_2 \end{bmatrix}$$

For information on how to solve this, see this footnote¹⁰. This same matrix formulation holds true for a 3D beam element.

The following is on how to solve *nonlinear* problems:

When we have a nonlinear effect (such as the stiffness increasing as deformation is increased), this relationship (Matrix Formulation $\{F\} = [K_e]\{d\}$) no longer holds true. Because these properties change, the equilibrium equations also change as the structure deforms, therefore you aren't solving the same structure you began with. The relationship between force and displacement is no longer proportional.

Its necessary to have this relationship relating forces and displacements to solve a model because the forces applied directly relate to how much a beam displaces/how much stress it undergoes. For nonlinear effects, A FEM will use the Newton-Raphson method to solve for the displacements, and that's why it requires the tangent stiffness (which is the derivative relationship between forces and displacements), represented by:

$$\frac{\partial f_{int}}{\partial u} \quad (28)$$

For more information on this method, see *Newton-Raphson Method* in Part 1 of this document. This allows you to find the displacements in a situation in which it is impossible to find the exact solutions every time. For a linear system, the solver can simply solve the relationship $\{F\} = [K_e]\{d\}$ since the stiffness of the beam doesn't vary. For nonlinear effects; however, the components of the stiffness matrix are variable at every point, so you require something more to solve it.

¹⁰<https://teachbooks.tudelft.nl/computational-modelling/introduction/matrix.html>, section 2.4 "Finite Element Implementation"

ix. Path Following Techniques and Their Differences There are two path following techniques that will allow you to solve a problem: Load Control and Displacement Control. Both require a nonlinear relation between $f_{int}(a)$ and $K(a) = \frac{\partial f_{int}}{\partial a}$. The point of path following is to allow you to solve the system to find the equilibrium points (where $W_{int} = W_{ext}$ in situations where the Newton-Raphson Method wouldn't provide enough information. For path following, you must choose one specific degree of freedom to analyze. For example, if you choose the vertical displacement, you would find the path curve of a certain point in the beam's vertical displacement (i.e. the entire curve is a function of only a single point 2 meters from the beam's node). Each point in the path curve represents the displacement (or whatever degree of freedom you chose) relative to a specific externally applied force/work, where the force/work function is the input.

In load control the goal is to gradually increase f_{ext} in a number of increments. Within each increment, we solve for equilibrium in an iterative way using Newton's method. It is therefore a so-called incremental-iterative approach, the main steps of which are shown below:

Load control

Require: Nonlinear relation $\mathbf{f}_{int}(\mathbf{a})$ with $\mathbf{K}(\mathbf{a}) = \frac{\partial \mathbf{f}_{int}}{\partial \mathbf{a}}$

Initialize: $n = 0, \mathbf{a}^0 = \mathbf{0}$

- **while** $n < \text{number of time steps}$:
 - Get new external force vector: $\mathbf{f}_{ext}^{n+1}$
 - Initialize new solution at old one: $\mathbf{a}^{n+1} = \mathbf{a}^n$
 - Compute internal force and stiffness: $\mathbf{f}_{int}^{n+1}(\mathbf{a}^{n+1}), \mathbf{K}^{n+1}(\mathbf{a}^{n+1})$
 - Evaluate residual: $\mathbf{r} = \mathbf{f}_{ext}^{n+1} - \mathbf{f}_{int}^{n+1}$
 - **while** $|\mathbf{r}| > \text{tolerance}$:
 - Solve linear system of equations: $\mathbf{K}^{n+1} \Delta \mathbf{a} = \mathbf{r}$
 - Update solution: $\mathbf{a}^{n+1} = \mathbf{a}^{n+1} + \Delta \mathbf{a}$
 - Compute internal force and stiffness: $\mathbf{f}_{int}^{n+1}(\mathbf{a}^{n+1}), \mathbf{K}^{n+1}(\mathbf{a}^{n+1})$
 - Evaluate residual: $\mathbf{r} = \mathbf{f}_{ext}^{n+1} - \mathbf{f}_{int}^{n+1}$
 - Go to the next time step: $n = n + 1$

Return: Set of internal forces and displacements (equilibrium path)

Figure 1: Details the process used for Load Control.¹¹

Load control is straightforward and suitable to a number of real-world applications. Nevertheless, it can lead to convergence issues and not be able to trace a full equilibrium path depending on the shape of the path. One example of such a situation is shown below:

¹¹https://teachbooks.tudelft.nl/computational-modelling/solid_nonlinear/solution_algorithms.html,section5.3

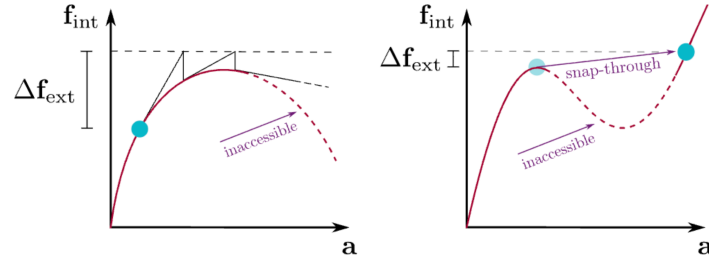


Figure 2: Tracing an equilibrium path with non-monotonic loading using load control.¹²

For equilibrium paths containing softening behavior, load control might lead to divergence (left) or to large jumps in solution (right) made difficult by a poor initial guess for a . Even if equilibrium is found, part of the path becomes effectively inaccessible.

An alternative to overcome this difficulty is to employ Dirichlet constraints and switch to a displacement control approach, the main steps of which are shown below:

Displacement control

Require: Nonlinear relation $\mathbf{f}_{\text{int}}(\mathbf{a})$ with $\mathbf{K}(\mathbf{a}) = \frac{\partial \mathbf{f}_{\text{int}}}{\partial \mathbf{a}}$

Initialize: $n = 0$, $\mathbf{a}^0 = \mathbf{0}$, partition DOFs into *free* (f) and *constrained* (c)

- **while** $n < \text{number of time steps}$:
 - Get new external force vector: $\mathbf{f}_{\text{ext}}^{n+1}$
 - Initialize new solution at old one: $\mathbf{a}^{n+1} = \mathbf{a}^n$
 - Compute internal force and stiffness: $\mathbf{f}_{\text{int}}^{n+1}(\mathbf{a}^{n+1})$, $\mathbf{K}^{n+1}(\mathbf{a}^{n+1})$
 - Constrain $\mathbf{K}^{n+1}$ so that $\Delta \mathbf{a}_c = \bar{\mathbf{a}}^{n+1} - \bar{\mathbf{a}}^n$
 - Evaluate residual at free DOFs: $\mathbf{r} = -\mathbf{f}_{\text{int},f}^{n+1}$
 - **while** $|\mathbf{r}| > \text{tolerance}$:
 - Solve linear system of equations: $\mathbf{K}^{n+1} \Delta \mathbf{a} = \mathbf{r}$
 - Update solution: $\mathbf{a}^{n+1} = \mathbf{a}^{n+1} + \Delta \mathbf{a}$
 - Compute internal force and stiffness: $\mathbf{f}_{\text{int}}^{n+1}(\mathbf{a}^{n+1})$, $\mathbf{K}^{n+1}(\mathbf{a}^{n+1})$
 - Evaluate residual at free DOFs: $\mathbf{r} = -\mathbf{f}_{\text{int},f}^{n+1}$
 - Constrain $\mathbf{K}^{n+1}$ so that $\Delta \mathbf{a}_c = \mathbf{0}$
 - Go to the next time step: $n = n + 1$

Return: Set of internal forces and displacements (equilibrium path)

Figure 3: Details the process used for Displacement Control.¹³

Note above that f_{ext} at the free nodes, and the residual therefore only contains f_{int} . Naturally, external loads are indirectly applied at constrained nodes in order to move them.

¹²https://teachbooks.tudelft.nl/computational-modelling/solid_nonlineair/solution_algorithms.html,section5.3

¹³https://teachbooks.tudelft.nl/computational-modelling/solid_nonlineair/solution_algorithms.html,section5.3

Since the equations linked to those DOFs are effectively eliminated from the system, these incurred forces do not appear in the solution scheme but can nevertheless be found during postprocessing by looking at $f_{int,c}$ (which, due to equilibrium, cannot be zero). Finally, note that since we are solving for Δa and not a , $\Delta_{a,c}$ must be constrained to zero after the first iteration of each step: a must already be at the prescribed value after the first linearized solve.

Engineering Strain vs True Strain Represented by ϵ , strain is the ratio of the deformation a beam/structure is experiencing over the length of the beam. This allows us to find the strain as a material property, independent of the length of the beam. Engineering strain can be found by:

$$\epsilon = \frac{\Delta L}{L_0} \quad (29)$$

where L_0 is the undeformed length of the beam and ΔL is the deformed length of the beam. This assumes that the beam's stiffness is going to remain constant throughout the deformation process (it won't stiffen or soften as the beam is bent). This is useful as it allows you to use the ratio:

$$\sigma = E\epsilon \quad (30)$$

during elastic deformation. The beam's stiffness will change during plastic deformation, and this relationship no longer holds true. We can see this in Figure 4, where the stiffness begins to change once plastic/large deformations begin to occur. The relationship (Eq. 30) holds true and is linear until ϵ is approximately equal to 0.1. True strain is better suited for when this nonlinear deformation begins to occur.

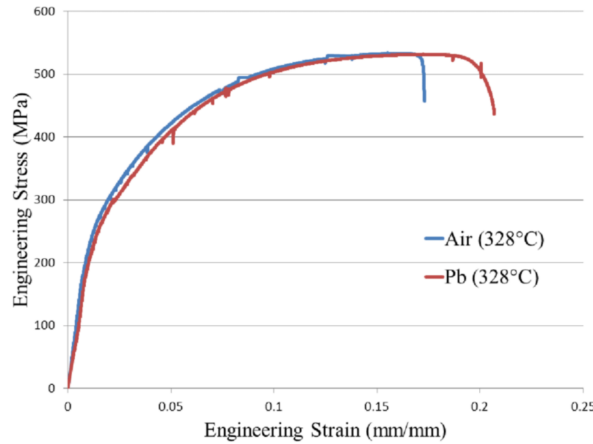


Figure 4: Engineering stress-strain curves for 1018 steel in air and molten lead both at 328°C showing similar tensile behavior.¹⁵

¹⁵Engineering stress-strain curves for 1018 steel in air and molten lead, both at 328°C [Figure]. ResearchGate. <https://www.researchgate.net/figure/Engineering-stress-strain-curves-for-1018-steel->

True strain is represented by:

$$\epsilon_{true} = \ln\left(\frac{L}{L_0}\right) \quad (31)$$

where L is the original, undeformed length of the beam, and L is the current length of the beam. There are also materials that deform in a nonlinear manner, for which true strain might be best suited. Engineering and true strains diverge at high strains due to their differences; however, its important to understand that just because engineering isn't accurate under every circumstance doesn't mean its a bad model. Engineering strain is simple to find and enables us to find the relationship between stress and Young's modulus easily (see Eq. 30). It assumes small (generally elastic) deformations and negligible changes in geometry, and is highly accurate when conditions match these assumptions, becoming inaccurate only under larger deformations and geometry changes. Engineering strain is useful for finding the beam's ability to resist deformation as a material property instead of being dependent upon the geometry of the beam. In summary, engineering strain is not a bad model *if* you use it under the proper deformation conditions.

Note: The beam's stiffness k is related to strain through the equation:

$$k = \frac{EA}{L} \quad (32)$$

3 Exploration and Application

3.1

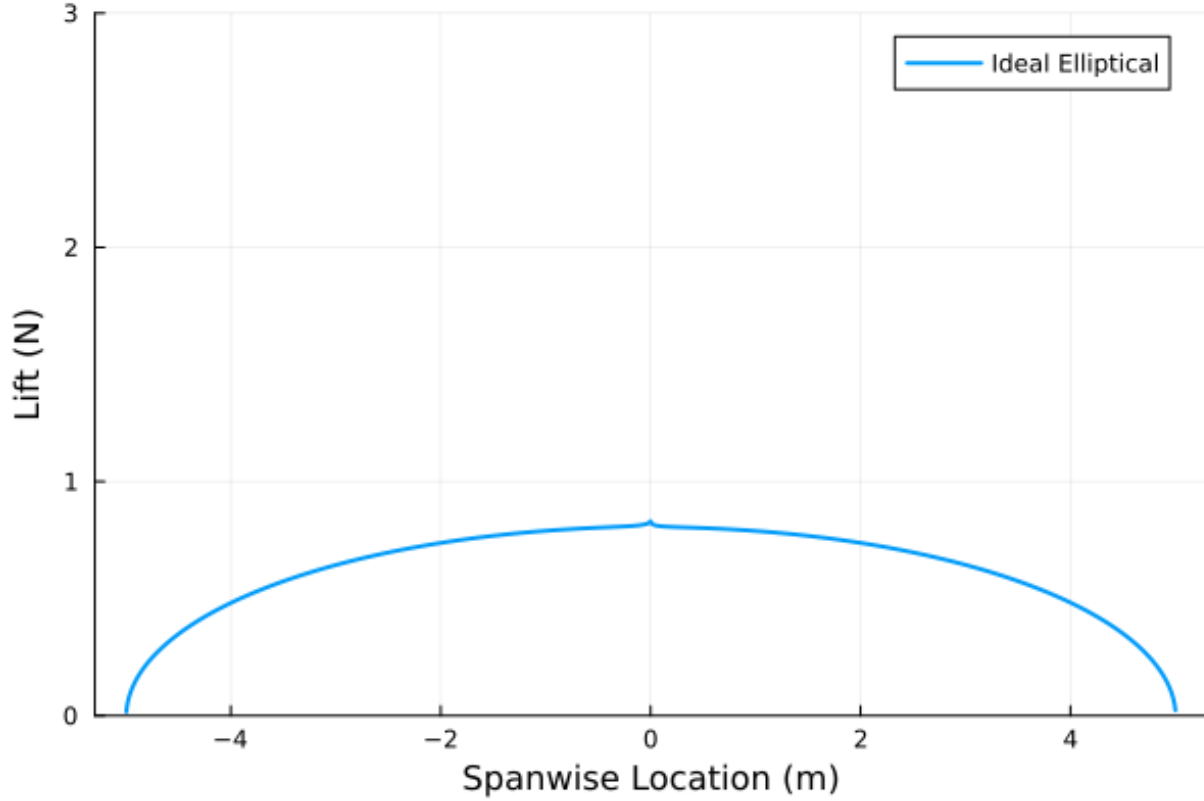


Figure 5: Results of 3.a, using $S_{ref} = 0.785$, $v = 10 \frac{m}{s}$, a half span of 5m, and a maximum chord of 0.1m

3.2

Note that in the Figure 6, all deformations not shown are all depicted by a horizontal line at 0. The above graphs represent the vertical displacement in the z-direction, the rotation about the z-axis, and the internal forces in the beam. The beam is represented as a simple cantilever beam, composed of steel, 0.05 meters wide and 0.01 meters tall. The units of the above graph are english units. See Appendix A for more details. I used the lift forces produced in 3.1 as a distributed load across the length of the beam, assuming the beam from 3.2 to be the wing. If you notice in the graph of vertical displacement, u_z continues to increase as you get further away from the origin, which is due to the beam's ability to move at the right end, whilst being restricted at the origin. The beam's stiffness does not vary as the deformation is increased; however, the distribution of forces applied does vary (See Figure 5). This is also the reason for the pattern you see in the rotation. Most of the

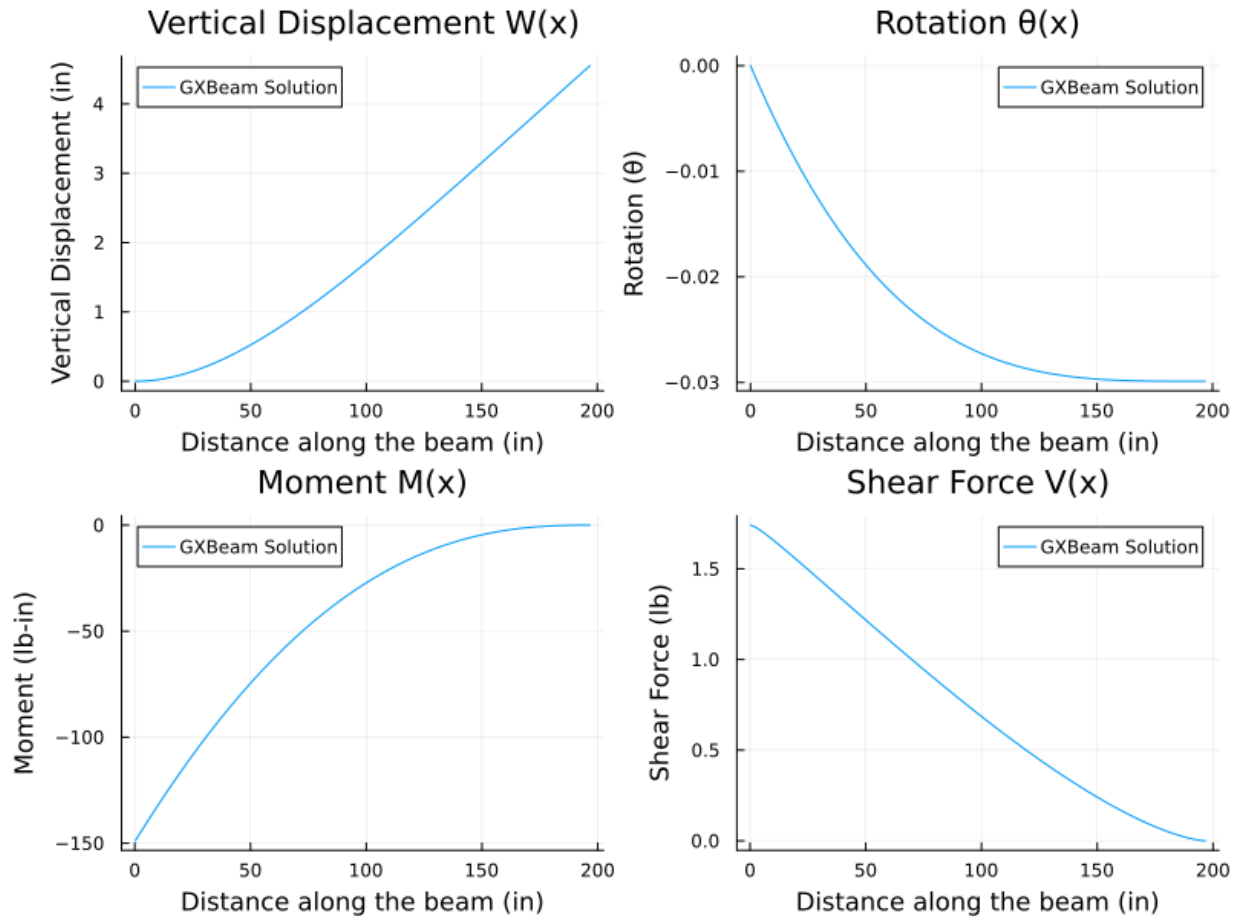


Figure 6: Results of 3.a ii, under linear assumptions in GXBeam, noting that all deformations and twists not represented are horizontal lines at 0.

change in rotation happens on the left half of the wing (closest to the origin) because that is where the majority of the force is applied. Little force is applied at the tips, causing relatively little rotation. The moment distribution also approaches 0 and stays there for a while due to the same reason (uneven load distribution, with little force applied at the tips). Notice the shear generally appears to have a linear slope that decreases as you approach the beam's tip (becoming more curved at the tip). This is because the shear is a sum of all the beam's resistance to vertical deformation. At the right tip, little deformation is occurring (because it's fixed); therefore, the beam must be opposing all the applied forces (if it is going to remain in a static state). As you move increasingly further away from the origin, the sum of all applied forces against which the beam must be imposed decreases until you reach the end of the beam, where the shear must equal 0. This curve begins to become more curved towards the right end because of the applied force's nonlinear relationship. This relationship between displacement and shear (as previously mentioned) is why these two graphs are almost mirror images of each other.

3.3

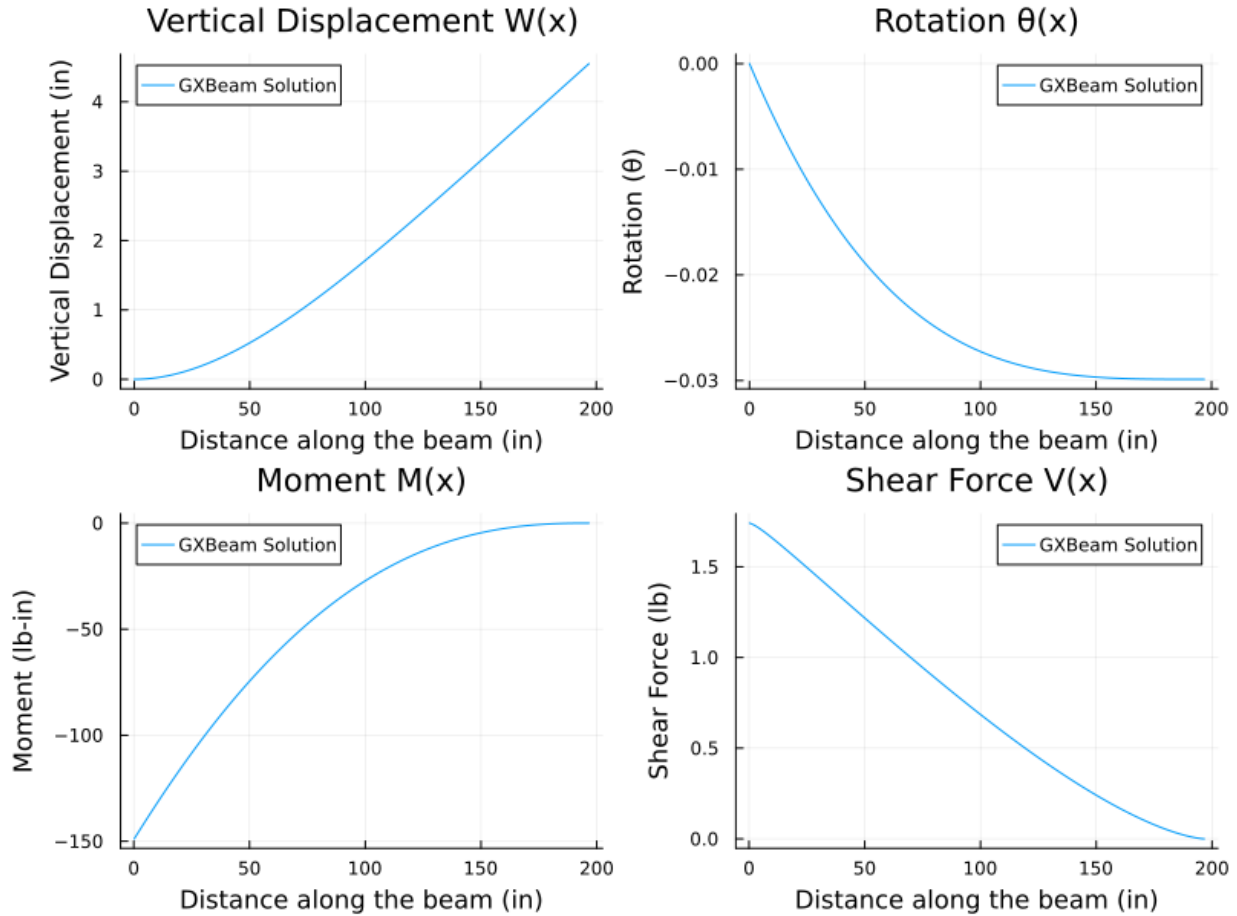


Figure 7: Results of 3.a iii under nonlinear assumptions in GXBeam, noting that all deformations and twists not represented are horizontal lines at 0.

The following is a description (also found in *Using Iterative Methods To Solve For Displacements*) about why there exist differences between the linear and non-linear iterative methods. The differences between Figures 6 and 7 are negligible due to the large number of spanwise panels used for this analysis (500); however, the differences are significantly more emphasized when there are fewer panels used. For more information on why this is, see below, as well as *Linearization* and *Newton-Raphson Method*.

There are two ways to solve for nodal displacements, one for nonlinear problems and one for linear problems. The following is for *linear* problems:

Each element within a Finite Element Method will contain geometric information, specifically the displacement u and the rotation θ . In 3D, it will contain 6 degrees of freedom: displacements u , v , and w , and rotations θ_x , θ_y , θ_z , in 2D it will only contain 4, and in 1D it will contain two degrees of freedom (u , θ_x). The purpose of these elements is to allow for bodies with more complex geometric features to be analyzed. Due to FEM's

inability to use a continuous representation, you must approximate segments of the beam (see ii. of this document for more information) to get an approximate geometry. Each element will have its own information (as listed previously), and the beam's information is depicted by the sum of all the elements' information.

The matrix formulation for a linear two-node beam element is given by:

$$\{F\} = [K_e]\{d\} \quad (33)$$

where F is the force matrix, K_e is the stiffness matrix for the element, and d is the matrix containing the degrees of freedom (with one column total and one row for each degree of freedom. For a 2D beam in bending, it would follow the form:

$$\{d\} = \begin{bmatrix} u_1 \\ \theta_1 \\ u_2 \\ \theta_2 \end{bmatrix}$$

since a 2D beam only has 4 degrees of freedom. This means that Eq. 15 would look something like:

$$\begin{bmatrix} f_1 \\ q\Delta x \\ q\Delta x \\ \frac{1}{2}q\Delta x + F \end{bmatrix} = \frac{EA}{\Delta x} \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \begin{bmatrix} u_1 \\ \theta_1 \\ u_2 \\ \theta_2 \end{bmatrix}$$

For information on how to solve this, see this footnote¹⁶. This same matrix formulation holds true for a 3D beam element.

The following is on how to solve *nonlinear* problems:

When we have a nonlinear effect (such as the stiffness increasing as deformation is increased), this relationship (Matrix Formulation $\{F\} = [K_e]\{d\}$) no longer holds true. Because these properties change, the equilibrium equations also change as the structure deforms, therefore you aren't solving the same structure you began with. The relationship between force and displacement is no longer proportional.

Its necessary to have this relationship relating forces and displacements to solve a model because the forces applied directly relate to how much a beam displaces/how much stress it undergoes. For nonlinear effects, A FEM will use the Newton-Raphson method to solve for the displacements, and that's why it requires the tangent stiffness (which is the derivative relationship between forces and displacements), represented by:

$$\frac{\partial f_{int}}{\partial u} \quad (34)$$

For more information on this method, see *Newton-Raphson Method* in Part 1 of this document. This allows you to find the displacements in a situation in which it is impossible

¹⁶<https://teachbooks.tudelft.nl/computational-modelling/introduction/matrix.html>, section 2.4 "Finite Element Implementation"

to find the exact solutions every time. For a linear system, the solver can simply solve the relationship $\{F\} = [K_e]\{d\}$ since the stiffness of the beam doesn't vary. For nonlinear effects; however, the components of the stiffness matrix are variable at every point, so you require something more to solve it.

Appendix A

Listing 1: Simulation Code

```
1 using FLOWMath
2 using LinearAlgebra
3 using Plots
4 using VortexLattice
5 using GXBeam
6 using Interpolations
7 bref = 10 # reference span
8 n = 500 # number of spanwise panels
9 xle = zeros(n) # no sweep
10     = range(0, /2; length=n)
11 b_half = bref*0.5
12 yle = b_half .* sin.( )
13 zle = zeros(n)
14 c_root = 0.1
15 chord = c_root .* sqrt.(1 .- (yle ./ b_half).^2)
16 theta = zeros(n)
17 phi = zeros(n)
18 fc = fill((xc) -> 0.0, n)
19 # camberline function for each section (y/c = f(x/c))
20 #This function describes the camberline of the airfoil sections
    along the span of the wing, creating the upper and
21 #lower edges of the airfoils. This function is set to zero (the
    first number), indicating that the wing has no camber
22 #and is a flat plate. The two indicates that there are two
    sections (root and tip) for which this function is defined
23 #(if xle, yle, & zle had three points, you would increase this
    second digit to be three).
24
25 ns = n # number of spanwise panels. Each panel is a trapezoidal
    shape bounded by the chords at the inner and outer
26 #edges, as well as the leading and trailing edges of the wing.
    Each panel has a different shape.
27 nc = 1 # number of chordwise panels. Similar to spanwise panels,
    but in the chordwise direction.
28 spacing_s = Uniform() # spanwise discretization scheme. This
    scheme uses a sine function to distribute the panels along
29 #the span of the wing, resulting in a higher concentration of
    panels near the wing root and tip, where aerodynamic
30 #effects are the greatest. In other words, the panels closest to
    the root and tips are narrower than those in between
31 #them (in the spanwise direction).
```

```

32 spacing_c = Uniform() # chordwise discretization scheme. Insted of
    concentrating the panels near the edges (See
33 #Sine function above), the panels are uniform throughout the wing
    (in the direction of the chord line).
34
35 grid, ratio = wing_to_grid(xle, yle, zle, chord, theta, phi, ns,
    nc;
36 fc = fc, spacing_s=spacing_s, spacing_c=spacing_c, mirror=false)#
    This combines all the factors included in the ()
37 #in order to create a meshed, 3-D shape of the wing. It stores
    this shape in the variable "grid", which is used later
38 #in the code to perform aerodynamic analysis. The "ratio" variable
    is a vector storing chorwise panel stretching ratios
39 #(the shape of the panels in the chordwise direction). This is
    used to compute the aerodynamic influence matrices
40 #and the scale vortex strengths during the analysis. The mirror
    option is set to true, meaning that the wing is mirrored
41 #across the aircraft's centerline to create a full wing (instead
    of just a half-wing).
42
43 grids = [grid] #list of grids in the system (only one wing in this
    case). It simply stores the grid created above
44 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
45 #surfaces)
46 ratios = [ratio] #list of ratios in the system (only one wing in
    this case). It simply stores the grid created above
47 #in a vector(useful for organizing and managing multiple grids in
    a more complex analysis involving several lifting
48 #surfaces)
49 system = System(grids; ratios) # Combines grids and ratios in a
    System object, which is a data structure used to store
50 #and manage all the lifting surfaces in the analysis. This object
    is essential for performing aerodynamic calculations.
51
52 Sref = 0.785 # reference area
53 cref = 0.1 # reference chord
54 rref = [0.50, 0.0, 0.0] # reference location for rotations/moments
    (typically the center of gravity)
55 Vinf = 10.0 # reference velocity (magnitude)
56 ref = Reference(Sref, cref, bref, rref, Vinf) #Creates a Reference
    object that stores the reference parameters used
57 #in the aerodynamic analysis.
58
59 alpha = 1.0*pi/180 # angle of attack

```

```

60 beta = 0.0 # sideslip angle. The angle between the aircrafts
    forward pointing axis and the actual direction the air
61 #is coming from. When you set beta = 0.0, you are telling the
    solver the aircraft has no sideways motion relative to
62 #the airflow.
63 Omega = [0.0, 0.0, 0.0] # rotational velocity around the reference
    location (rref). This vector represents the angular
64 #velocity of the aircraft in three-dimensional space, with
    components along the x, y, and z axes. Setting it to
65 #[0.0, 0.0, 0.0] indicates that the aircraft is not rotating.
66 fs = Freestream(Vinf, alpha, beta, Omega) #Combines the freestream
    parameters into a Freestream object, which is used
67 #to define the incoming airflow conditions for the aerodynamic
    analysis.
68
69 symmetric = false #When set to false, the solver considers both
    sides of the aircraft separately, allowing for asymmetric
70 #flow conditions and forces. This is important for accurately
    capturing the aerodynamic behavior of the aircraft,
71 #especially during maneuvers (anytime theres a rolling and yawing
    moment) or in the presence of crosswinds. When set
72 #to false, the solver does not assume symmetry about the aircraft's
    centerline and cancel out forces and moments.
73
74 steady_analysis!(system, ref, fs; symmetric)
75 #This function performs a steady-state aerodynamic analysis on the
    lifting surfaces defined in the "system" object,
76 #using the reference parameters in the "ref" object and the
    freestream conditions in the "fs" object (Simply put,
77 #it takes all the information found above, and performed an
    aerodynamic analysis). The analysis assumes that the flow
78 #conditions are constant over time (steady) and that the aircraft
    is not symmetric about its centerline (symmetric=false).
79 #The results of the analysis, including aerodynamic forces and
    moments, are stored within the "system" object for further
80 #evaluation
81 CF, CM = body_forces(system; frame=Wind()) #This creates the force
    and moment vectors in the wind frame of reference.
82 #Of the force vector, the x-component is the drag force, the y-
    component is the side force, and the z-component is
83 #the lift force. Of the moment vector, the x-component is the
    rolling moment, the y-component is the pitching moment,
84 #and the z-component is the yawing moment.
85
86 # extract aerodynamic forces from the vectors created above.

```

```

87 CD, CY, CL = CF
88 Cl, Cm, Cn = CM
89
90 CDiff = far_field_drag(system) # Computes the drag contribution from
    the far-field induced drag, which is the drag caused by the
91 #vortices shed into the wake behind the lifting surfaces. This
    function calculates the far-field drag based on the vortex
    strengths
92 #and wake geometry determined during the steady-state analysis.
    The result is stored in the variable "CDiff".
93 #Simply put --> It calculated the coefficient of induced drag.
94
95 # calculate lifting line coefficients
96 cf, cm = lifting_line_coefficients(system; frame=Body()) #cf is
    the calculated section force coefficients along the
97 #span, and cm is the calculated section moment coefficients along
    the span. They are distributions and not total forces.
98 #frame=body means that its using the reference frame of the
    aircraft body (as opposed to wind frame used earlier).
99
100 Gamma = system.          #This function retrieves the vortex strengths
    for the lifting surfaces defined in the
101 filtered_Gamma = []
102 density = 0.38
103 total_lift_elyptical = 0
104 for i in 1:length(Gamma)
105     if i % 1 == 0
106         Gammepsu = Gamma[i]
107         Lpsu1 = 0.5 * Gammepsu * density * Vinf^2 * Sref
108         global total_lift_elyptical += Lpsu1
109         push!(filtered_Gamma, Lpsu1)
110     end
111 end
112
113 xax = [39.3701 * grid[2,1,i+1] for i in 1:length(filtered_Gamma)]
114 plot!(xax, filtered_Gamma, label="Elliptical Wing", linewidth=1)
115 #println("Total Lift for Tapered Chord Wing: ",
    total_lift_elyptical)
116 # println("Total Lift for an Elliptical Wing: ", trapz(xax,
    filtered_Gamma))
117
118 # L0 = maximum(filtered_Gamma)
119 # b = bref
120 # ideal_elyptical = []
121 # for i in 2:length(grid[2,1,:])

```

```

122 #      liftpsu = L0 * sqrt(1 - ((2 * grid[2,1,i])/(b))^2)
123 #      push!(ideal_elyptical, liftpsu)
124 # end
125
126
127 #plot!(xax, ideal_elyptical, label="Ideal Elyptical Wing",
128       linewidth=3)
129 # pe1 = plot(xax, ideal_elyptical, ylims=(0, .55), label="Ideal
130       Elliptical", linewidth=2, xlabel="Spanwise Location (m)",
131       ylabel="Lift per Unit Span (N/m)")
132 # plot!(xax, filtered_Gamma, label="Actual Elliptical", linewidth
133       =2)
134 # plot(pcc, pe1, layout=(2,1))
135 plot(xax, filtered_Gamma, ylims=(0, 2.5), label="Ideal_Elliptical"
136       , linewidth=2, xlabel="Spanwise_Location_(m)", ylabel="Lift_(N)"
137       ")
138 # println("Efficiency - Elyptical: ", CF[3]/CF[1]) #Lift to Drag
139       Ratio
140 # AR = (bref^2)/Sref
141 # println((CF[3]^2)/ (pi * CDiff * AR))
142
143 #println(CF[3])
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158

```

```

159
160
161
162
163
164
165
166
167
168
169
170
171
172 # straight section of the beam
173 L_b1 = b_half/2 * 39.3701 # length of straight section of the
    beam in inches
174 r_b1 = [0, 0, 0] # starting point of straight section of the beam
175 nelem_b1 = n      2 # number of elements in the straight section of
    the beam
176 lengths_b1, xp_b1, xm_b1, Cab_b1 = discretize_beam(L_b1, r_b1,
    nelem_b1)
177
178 # swept section of the beam
179 L_b2 = b_half/2 * 39.3701 # length of swept section of the beam
180 r_b2 = [L_b1, 0, 0] # starting point of swept section of the beam
181 nelem_b2 = n      2 # number of elements in swept section of the
    beam
182 e1 = [1, 0, 0] # axis 1
183 e2 = [0, 1, 0] # axis 2
184 e3 = [0, 0, 1] # axis 3
185 frame_b2 = hcat(e1, e2, e3) # transformation matrix from local to
    body frame
186 lengths_b2, xp_b2, xm_b2, Cab_b2 = discretize_beam(L_b2, r_b2,
    nelem_b2;
187     frame = frame_b2)
188
189
190 # combine elements and points into one array
191 nelem = nelem_b1 + nelem_b2 # total number of elements
192 points = vcat(xp_b1, xp_b2[2:end]) # all points in our assembly
193 start = 1:nelem_b1 + nelem_b2 # starting point of each beam
    element in our assembly
194 stop = 2:nelem_b1 + nelem_b2 + 1 # ending point of each beam
    element in our assembly

```

```

195 lengths = vcat(lengths_b1, lengths_b2) # length of each beam
    element in our assembly
196 midpoints = vcat(xm_b1, xm_b2) # midpoint of each beam element in
    our assembly
197 Cab = vcat(Cab_b1, Cab_b2) # transformation matrix from local to
    body frame
198                                     # for each beam element in our assembly
199
200
201
202 # elem_lengths = lengths # already in inches
203 elem_mid = [midpoints[i][1] for i in 1:nelem]
204 q_lift = [(filtered_Gamma[i] / lengths[i]) * 0.005708 for i in 1:
    nelem]
205 #).005708 or 0.22. I think it should be 0.005708
206 # q_lift = []
207
208 # for i in 1:nelem
209 #     # find nearest aerodynamic panel
210 #     idx = argmin(abs.(xax .- elem_mid[i]))
211
212 #     # convert N         lb
213 #     L_N = filtered_Gamma[idx]
214 #     L_lb = L_N * 0.224809
215
216 #     # divide by element length (inches)
217 #     q = L_lb / lengths[i]
218
219 #     push!(q_lift, q)
220 # end
221
222
223 qliftplot = plot(elem_mid, q_lift, label="Distributed_Lift",
    xlabel="Distance_along_the_beam(in)", ylabel="Lift_per_Unit_
    Length(N/in)", title="Distributed_Lift_Along_the_Beam")
224
225
226
227 # cross section
228 h = 0.3937 # inch
229 w = 1.9685 # inch
230
231 # material properties
232 E = 29000000 # lb/in^2
233     = 0.28

```

```

234     = 0.284 # lb sec^2/in^4
235
236 # shear and torsion correction factors
237 ky = 0.83
238 kz = 0.83
239 kt = 1
240
241 A = h*w
242 Iyy = w*h^3/12
243 Izz = w^3*h/12
244 J = Iyy + Izz
245
246 # apply corrections
247 Ay = A/ky
248 Az = A/kz
249 Jx = J/kt
250
251 G = E/(2*(1+ ))
252
253 compliance = fill(Diagonal([1/(E*A), 1/(G*Ay), 1/(G*Az), 1/(G*Jx),
254     1/(E*Iyy),
255     1/(E*Izz)]), nelem)#creates a 6x6 compliance matrix for each
256     element in the assembly, where the diagonal
257     #entries correspond to the inverse of the stiffness of the
258     element in each degree of freedom. The
259     #compliance matrix is used to relate forces and moments to
260     displacements and rotations in the beam
261     #elements.
262
263
264 mass = fill(Diagonal([ *A, *A, *A, *J, *Iyy, *Izz]),
265     nelem)#creates a 6x6 mass matrix for each element
266 #in the assembly, where the diagonal entries correspond to the
267 mass of the element in each degree of freedom.
268 #The mass matrix is used to relate accelerations to forces and
269 moments in the beam elements.
270
271
272 assembly = Assembly(points, start, stop;
273     compliance = compliance,
274     mass = mass,
275     frames = Cab,
276     lengths = lengths,
277     midpoints = midpoints)

```

```

272 # mkpath("rotating-geometry")
273 # write_vtk("rotating-geometry/rotating-geometry", assembly)
274
275 # ADDING POINT MASSES
276 # m = 10 # mass
277 # p = zeros(3) # relative location
278 # J = zeros(3,3) # inertia matrix (about the point mass center of
    gravity)
279
280 # # create dictionary of point masses
281 # point_masses = Dict(
282 #     nelem+1 => PointMass(m, p, J)
283 # )
284
285 # DISTRIBUTED LOADS FROM AERODYNAMIC LIFT (using right-end value)
286 distributed_loads = Dict{Int64, DistributedLoads{Float64}}{()}
287 for ielem in 1:nelem
288     q = q_lift[ielem]    # lb/in, force per unit length for this
        element
289     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
290         fz = (s) -> q    # constant load over the element
291     )
292 end
293
294
295 # # DISTRIBUTED LOADS THAT ROTATE WITH THE BEAM
296 # distributed_loads = Dict{Int64, DistributedLoads{Float64}}{()}
297 # for ielem in 1:nelem
298 #     distributed_loads[ielem] = DistributedLoads(assembly, ielem;
299 #         fz_follower = (s) -> 1)
300 # end
301
302 # Defining PRESCRIBED CONDITIONS
303 # create FIXED BOUNDARY CONDITION ON THE LEFT SIDE (ROOT) OF THE
    BEAM-->
304 prescribed_conditions = Dict(
305     # left support: fix all translations, block rotation about
        beam axis
306     1 => PrescribedConditions(ux = 0, uy = 0, uz = 0, theta_x = 0,
        theta_y = 0, theta_z = 0)
307 )
308
309 # PREALLOCATING MEMORY FOR OUR ANALYSIS(CAN USE StaticSystem,
    DynamicSystem, or ExpandedSystem(when using the mass matrix for
    differential equations))

```

```

310 system = DynamicSystem(assembly)
311
312
313
314
315
316
317
318
319 # Performing a Steady State ANALYSIS
320 rpm = [0.0]
321
322 # linear_states = Vector{AssemblyState{Float64}}(undef, length(rpm
    ))
323 # for i = 1:length(rpm)
324
325 #     # global frame rotation
326 #     w0 = [0, 0, rpm[i]*(2*pi)/60]
327
328 #     # perform linear steady state analysis
329 #     _, state, converged = steady_state_analysis!(system,
    assembly,
330 #         angular_velocity = w0,
331 #         prescribed_conditions = prescribed_conditions,
332 #         distributed_loads = distributed_loads,
333 #         linear = true)
334
335 #     # save result
336 #     linear_states[i] = state
337
338 # end
339
340 # reset_state!(system)
341
342 nonlinear_states = Vector{AssemblyState{Float64}}(undef, length(
    rpm))
343 for i = 1:length(rpm)
344
345     # global frame rotation
346     w0 = [0, 0, rpm[i]*(2*pi)/60]
347
348     # perform nonlinear steady state analysis
349     _, state, converged = steady_state_analysis!(system, assembly,
    angular_velocity = w0,
350     distributed_loads = distributed_loads,

```

```

352         prescribed_conditions = prescribed_conditions)
353
354     # save result
355     nonlinear_states[i] = state
356
357 end
358 # ^^Linear solution assumes small deformations and rotations,
    while
359 #nonlinear solution accounts for large deformations and rotations.
360 #The linear solution may not be accurate at higher rotation speeds
361 #where the beam experiences significant deformation, while the
    nonlinear
362 #solution provides a more accurate representation of the beam's
    behavior
363 #under these conditions.
364 # ^^Linear assumes no geometric stiffening, no coupling between
    bending,
365 #torsion and axial stretch, and centrifugal forces are linearized
    .
366 #Nonlinear accounts for all of these effects, which become more
    significant
367 #at higher rotation speeds.
368 #Nonlinear will update stiffness as the beam deflects
369
370 # i = 1 # first RPM case
371 # # vertical displacement u_z at every node
372 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
    points)]
373 # # x coordinates of each node
374 # xvaluesnumerical = [points[n][1] for n in 1:length(points)]
375 # println(uz_all)
376
377
378 # #####LINEAR
379 # i = 1 # for example, the 1st RPM case
380 # uz_all = [linear_states[i].points[n].u[3] for n in 1:length(
    points)]
381 # uy_all = [linear_states[i].points[n].u[2] for n in 1:length(
    points)]
382 # ux_all = [linear_states[i].points[n].u[1] for n in 1:length(
    points)]
383 # xvaluesnumerical = [points[aa][1] for aa in 1:length(points)]
384 # thetaxGX = [linear_states[i].points[n].theta[2] for n in 1:
    length(points)]
385 # #print(length(linear_states[i].elements))

```

```

386 # #println(linear_states[i].elements[16].Mi)
387 # moments = [linear_states[i].elements[n].Mi[2] for n in 1:length(
    linear_states[i].elements)]
388 # xvalues12 = [midpoints[n][1] for n in 1:length(linear_states[i].
    elements)]
389 # shears = [linear_states[i].elements[n].Fi[3] for n in 1:length(
    linear_states[i].elements)]
390
391 #####NONLINEAR
392 i = 1 # for example, the 1st RPM case
393 uz_all = [nonlinear_states[i].points[n].u[3] for n in 1:length(
    points)]
394 uy_all = [nonlinear_states[i].points[n].u[2] for n in 1:length(
    points)]
395 ux_all = [nonlinear_states[i].points[n].u[1] for n in 1:length(
    points)]
396 xvaluesnumerical = [points[aa][1] for aa in 1:length(points)]
397 thetaxGX = [nonlinear_states[i].points[n].theta[2] for n in 1:
    length(points)]
398 #print(length(linear_states[i].elements))
399 #println(linear_states[i].elements[16].Mi)
400 moments = [nonlinear_states[i].elements[n].Mi[2] for n in 1:length
    (nonlinear_states[i].elements)]
401 xvalues12 = [midpoints[n][1] for n in 1:length(nonlinear_states[i]
    ].elements)]
402 shears = [nonlinear_states[i].elements[n].Fi[3] for n in 1:length(
    nonlinear_states[i].elements)]
403
404
405 # wx = []
406 # thetax = []
407 # xvalues = []
408 # Mx = []
409 # Vx = []
410 # for i in 1:length(points)
411 #     q0 = 1
412 #     #E = 1.06e7 # lb/in^2
413 #     x = points[i][1]
414 #     L = b_half/2 * 39.3701
415 #     #println(x)
416 #     wxpart = ((q0 * L^4) / (24 * E * Iyy)) * ((x/L) - 2*(x^3/L
    ^3) + (x^4/L^4))
417 #     thetaxpart = -(q0*L^3) / (24 * E * Iyy) * (1 - 6*(x^2/L^2) +
    4*(x^3/L^3))
418 #     Mxpart = ((q0*L^2) / 2) * ((x/L) - (x^2/L^2))

```

```

419 #      Vxpart = ((q0*L) / 2) * (1 - 2*(x/L))
420 #      push!(wx, wxpart)
421 #      push!(xvalues, x)
422 #      push!(thetax, thetaxpart)
423 #      push!(Mx, Mxpart)
424 #      push!(Vx, Vxpart)
425 # end
426
427 # pwx = plot(xvalues, wx, ylabel="Displacement (in)", xlabel="
      Distance along the beam (in)", title="Vertical Displacement W(x
      )", label="Analytical Solution")
428 pwxGXz = plot(xvaluesnumerical, uz_all, label="GXBeam_Solution",
      xlabel="Distance_along_the_beam(in)", ylabel="Vertical_
      Displacement_(in)", title="Vertical_Displacement_W(x)")
429 pwxGXy = plot(xvaluesnumerical, uy_all, label="GXBeam_Solution",
      xlabel="Distance_along_the_beam(in)", ylabel="Displacement_
      along_the_y-axis(in)", title="Vertical_Displacement_W(x)")
430 pwxGXx = plot(xvaluesnumerical, ux_all, label="GXBeam_Solution",
      xlabel="Distance_along_the_beam(in)", ylabel="Displacement_
      along_the_x-axis(in)", title="X-axis_Displacement_W(x)")
431 # pthetax = plot(xvalues, thetax, ylabel="Rotation ( )", xlabel="
      Distance along the beam (in)", title="Rotation (x)", label="
      Analytical Solution")
432 pthetaxGX = plot(xvaluesnumerical, thetaxGX, label="GXBeam_
      Solution", xlabel="Distance_along_the_beam(in)", ylabel="
      Rotation_( )", title="Rotation_(x)")
433 # pMx = plot(xvalues, Mx, ylabel="Moment (lb-in)", xlabel="
      Distance along the beam (in)", title="Moment M(x)", label="
      Analytical Solution")
434 pMxGX = plot(xvalues12, moments, label="GXBeam_Solution", xlabel="
      Distance_along_the_beam(in)", ylabel="Moment_(lb-in)", title="
      Moment_M(x)")
435 # pVx = plot(xvalues, Vx, ylabel="Shear Force (lb)", xlabel="
      Distance along the beam (in)", title="Shear Force V(x)", label
      ="Analytical Solution")
436 pVxGX = plot(xvalues12, shears, label="GXBeam_Solution", xlabel="
      Distance_along_the_beam(in)", ylabel="Shear_Force_(lb)", title
      ="Shear_Force_V(x)")
437 plot(pwxGXz, pthetaxGX, pMxGX, pVxGX, layout=(2, 2), size
      =(800,600))

```